

PRACTICUM WORKSHEET: THE AMDAHL REALITY GAP

Empirical Speedup Fitting, Micro-Architectural Bottlenecks, and Loop Scheduling in OpenMP

Field	Entry
Student Full Name	Seiitkhan Zhannur
Student ID Number	230109009
Institutional Email / Git URL	230109009@sdu.edu.kz
Practicum Date & Time (Slot)	24 September 2026
Laptop Model & OS	Victus by HP Gaming Laptop 15-fa0xxx / Windows
CPU Model & Generation	12th Gen Intel(R) Core(TM) i5-12450H
Physical Cores (P/E breakdown)	8 (4 P-cores + 4 E-cores)
Logical SMT Threads (omp_get_max_threads)	12
Target Workload Size N	$N = 10,000,000 + (9009 \times 1,000) = 19,009,000$
L1 / L2 Cache Line Size	64 bytes

Phase 1: Machine Probing and Workload

Hardware information was captured in PowerShell and saved to hw_info.txt:

```
Get-CimInstance Win32_Processor | Select-Object Name, NumberOfCores,
NumberOfLogicalProcessors | Format-List
```

```
Name                : 12th Gen Intel(R) Core(TM) i5-12450H
NumberOfCores       : 8
NumberOfLogicalProcessors : 12
```

The i5-12450H is a hybrid CPU: 4 Performance cores (with SMT, 8 threads) plus 4 Efficiency cores (1 thread each), giving 8 physical cores and 12 logical threads. All programs were compiled with `gcc -O2 -fopenmp` and timed with `omp_get_wtime()`.

Timing protocol: Run 1 of every configuration is the cold run and is discarded. Each reported time is $T = (\text{Run 2} + \text{Run 3}) / 2$. In Phase 2 and Phase 3 every run was a separate program launch; in Phase 4 the program repeats each configuration three times internally.

Phase 2: Sequential Baseline

```
PS C:\Users\szhan\paralcomp> cd C:\Users\szhan\paralcomp\amdahl_lab
>> gcc -O2 -fopenmp collatz_seq.c -o collatz_seq.exe
>> .\collatz_seq.exe
N = 19009000
Max stopping time = 704
Checksum = 78178256
Execution time = 9.725999832 seconds
PS C:\Users\szhan\paralcomp\amdahl_lab> cd C:\Users\szhan\paralcomp\amdahl_lab
>> gcc -O2 -fopenmp collatz_seq.c -o collatz_seq.exe
>> .\collatz_seq.exe
N = 19009000
Max stopping time = 704
Checksum = 78178256
Execution time = 9.430999994 seconds
PS C:\Users\szhan\paralcomp\amdahl_lab> cd C:\Users\szhan\paralcomp\amdahl_lab
>> gcc -O2 -fopenmp collatz_seq.c -o collatz_seq.exe
>> .\collatz_seq.exe
N = 19009000
Max stopping time = 704
Checksum = 78178256
Execution time = 9.655999899 seconds
PS C:\Users\szhan\paralcomp\amdahl_lab>
```

Figure 1: Initial sequential runs (collatz_seq.exe, N = 19,009,000).

Item	Initial run (Figure 1)	Re-run (seq_log.txt)
N	19,009,000	19,009,000
Max stopping time	704	704
Checksum (sum of steps mod 1,000,000,007)	78178256	78178256
Run 1 (cold, discarded)	9.726 s	9.693 s
Run 2	9.431 s	9.124 s
Run 3	9.656 s	8.825 s
T_seq = (Run 2 + Run 3) / 2	9.5435 s	8.9745 s

Why there are two baselines. The initial T_seq (9.5435 s) was slower than the 1-thread OpenMP run in Phase 3 (T_1 = 8.4725 s) and gave S_emp(2) = 2.161, i.e. $p = 1.074$, which is impossible (p must lie in $[0, 1]$). I therefore re-measured the sequential program; the re-run gives T_seq = 8.9745 s. The three sequential runs drift downward (9.69, 9.12, 8.83 s), which may reflect turbo or thermal behaviour on a hybrid laptop CPU; I did not investigate the cause. Both sets of runs are reported above; the re-run is the T_seq used in the rest of this document.

Phase 3: Multi-Thread Scaling and Empirical Amdahl Fitting

The loop was parallelized with #pragma omp parallel for (default static schedule) with reduction(max:) and reduction(+:). Every thread count was launched three times as a separate process; the results are logged in phase3_log.txt. Max stopping time (704) and checksum (78178256) were identical for every run.

```
PS C:\Users\szhan\paralcomp\amdahl_lab> cd C:\Users\szhan\paralcomp\amdahl_lab
>> gcc -O2 -fopenmp collatz_seq.c -o collatz_seq.exe
>> .\collatz_seq.exe
Checksum = 78178256
>> gcc -O2 -fopenmp collatz_par.c -o collatz_par.exe
>> .\collatz_par.exe 19009000 2ahl_lab>
N = 19009000
Threads = 2
Max stopping time = 704
Checksum = 78178256
Execution time = 4.562999964 seconds
PS C:\Users\szhan\paralcomp\amdahl_lab> foreach ($k in 1,2,4,8) { foreach ($r in 1..3) { "k=$k run=$r" | Tee-Object -FilePath phase3_log.txt -Append; .\collatz_par.exe 19009000
$ $k | Tee-Object -FilePath phase3_log.txt -Append } }
k=1 run=1
N = 19009000
Threads = 1
Max stopping time = 704
Checksum = 78178256
Execution time = 8.688999891 seconds
k=1 run=2
N = 19009000
Threads = 1
Max stopping time = 704
Checksum = 78178256
Execution time = 8.483999968 seconds
k=1 run=3
N = 19009000
Threads = 1
Max stopping time = 704
Checksum = 78178256
Execution time = 8.461000204 seconds
k=2 run=1
N = 19009000
```

```

PS C:\Users\szhan\paralcomp\amdahl_lab> foreach ($k in 1,2,4,8) { foreach ($r in 1..3) { "k=$k run=$r" | Tee-Object -FilePath phase3_log.txt -Append; .\collatz_par.exe 19009000
$k | Tee-Object -FilePath phase3_log.txt -Append } }
k=4 run=2
N = 19009000
Threads = 4
Max stopping time = 704
Checksum = 78178256
Execution time = 2.230999947 seconds
k=4 run=3
N = 19009000
Threads = 4
Max stopping time = 704
Checksum = 78178256
Execution time = 2.302999973 seconds
k=8 run=1
N = 19009000
Threads = 8
Max stopping time = 704
Checksum = 78178256
Execution time = 1.296000004 seconds
k=8 run=2
N = 19009000
Threads = 8
Max stopping time = 704
Checksum = 78178256
Execution time = 1.269999981 seconds
k=8 run=3
N = 19009000
Threads = 8
Max stopping time = 704
Checksum = 78178256
Execution time = 1.343000174 seconds
PS C:\Users\szhan\paralcomp\amdahl_lab>

```

Figure 2: Parallel runs (screenshots for $k = 1, 2, 4, 8$; $k = 12$ and 16 are in phase3_log.txt).

Table 1: Measurement log and speedup

$S_{\text{emp}}(k) = T_1 / T_k$ with baseline $T_1 = 8.4725$ s; $S_{\text{theo}}(k)$ from Amdahl's law with $p = 0.957$; $\Delta(k) = S_{\text{theo}}(k) - S_{\text{emp}}(k)$. Run 1 is the discarded cold run. $k = 12$ is the maximum logical thread count; $k = 16$ is an extra oversubscription point.

k	Run 1 (cold) s	Run 2 (s)	Run 3 (s)	Avg T_k (s)	$S_{\text{emp}}(k)$	$S_{\text{theo}}(k)$	$\Delta(k)$
1	8.689	8.484	8.461	8.4725	1.000	1.000	0.000
2	4.243	4.373	4.460	4.4165	1.918	1.918	0.000
4	2.437	2.231	2.303	2.2670	3.737	3.547	-0.190
8	1.296	1.270	1.343	1.3065	6.485	6.164	-0.321
12 (max)	1.309	0.991	0.994	0.9925	8.537	8.174	-0.362
16	0.968	0.902	0.872	0.8870	9.552	9.767	+0.215

Derivation of the parallel fraction p

$$S(2) = 1 / [(1 - p) + p/2] \Rightarrow 1/S(2) = 1 - p/2 \Rightarrow p = 2 \times [1 - 1/S_{\text{emp}}(2)]$$

- **Baseline T_1 (used):** $S_{\text{emp}}(2) = 8.4725 / 4.4165 = 1.918$, so $p = 2 \times (1 - 1/1.918) = 0.957$. About 95.7% of the runtime is parallelizable and about 4.3% is sequential (thread start-up, reduction, loop overhead).
- **Baseline T_{seq} (not used):** $S_{\text{emp}}(2) = 8.9745 / 4.4165 = 2.032$, so $p = 1.016$. This is slightly superlinear and physically invalid; the excess is about 1.6%, smaller than the drift I observed across the three sequential runs (about 9%). I therefore used the 1-thread OpenMP time T_1 as the baseline, because it runs the same code path and its runs were very consistent (8.484 s and 8.461 s).

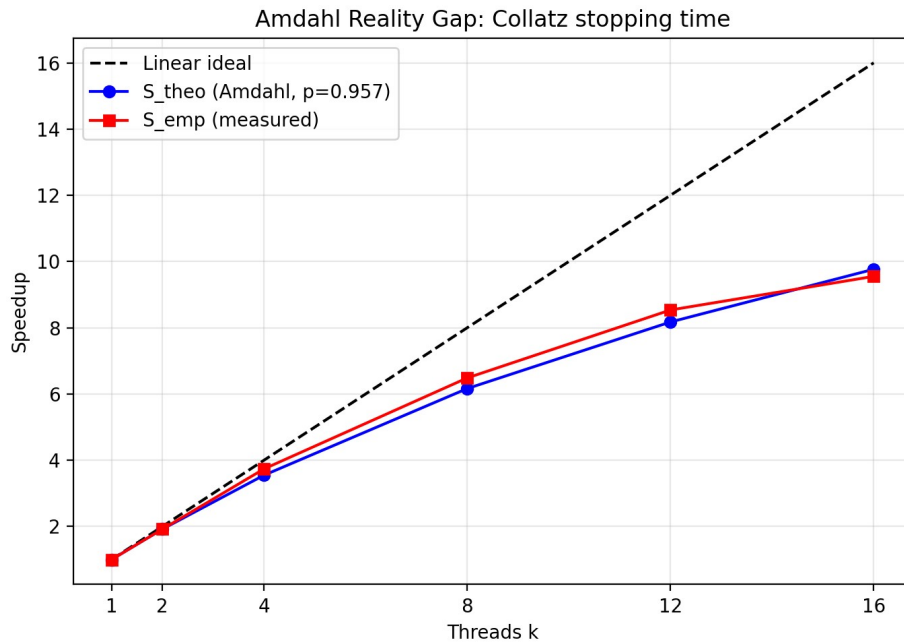


Figure 3: Speedup versus thread count (speedup_plot.png). Baseline T_1 , $p = 0.957$.

Reality gap

Measured speedup stays close to the fitted Amdahl curve: Delta is slightly negative for $k = 4, 8$ and 12 (measured slightly better than the fit) and positive only at $k = 16$ (+0.215). The large gap is between both curves and the linear ideal: at $k = 12$ the measured speedup is 8.54 instead of 12 (71%), and at $k = 16$ it is 9.55 instead of 16 (60%). The causes are discussed in Q2 and Q3. Because p is fitted from a single measurement at $k = 2$, the fitted curve is sensitive to noise in that one point.

Phase 4: Hardware Penalties and Micro-Architectural Experiments

All results below come from collatz_phase4.exe ($N = 19,009,000$). The task counts numbers in $[1, N]$ needing more than 100 steps; every variant and schedule returned the same count, 15,645,999, as well as max stopping time 704 and checksum 78178256.

```
>> .\collatz_phase4.exe 19009000 12 | Tee-Object -FilePath phase4_k12_log.txt
>> .\collatz_phase4.exe 19009000 12 | Tee-Object -FilePath phase4_k12_log.txt
N = 19009000, threads = 8, omp_get_max_threads = 12

=== Experiment A: False sharing ===
V1 naive hits[tid] threads=8 run1=1.1500 run2=0.9740 run3=1.0430 avg=1.0085 s | hits=15645999 | throughput=1.885e+07 iter/s
V2 padded struct threads=8 run1=0.9760 run2=1.0080 run3=0.9570 avg=0.9825 s | hits=15645999 | throughput=1.935e+07 iter/s
V2 reduction threads=8 run1=1.2530 run2=1.2840 run3=2.2620 avg=1.7730 s | hits=15645999 | throughput=1.072e+07 iter/s

=== Experiment B: Loop scheduling (8 threads) ===
static (default) run1=2.6400 run2=2.5530 run3=2.9920 avg=2.7725 s | thread busy min=2.7470 max=2.9900 (max/min=1.09) | max_steps=704 checksum=78178256 hits=15645999
static, 1000 run1=3.1880 run2=2.9510 run3=2.3030 avg=2.6270 s | thread busy min=2.2210 max=2.3030 (max/min=1.04) | max_steps=704 checksum=78178256 hits=15645999
dynamic, 100 run1=1.2160 run2=1.2320 run3=1.1990 avg=1.2155 s | thread busy min=1.1990 max=1.1990 (max/min=1.00) | max_steps=704 checksum=78178256 hits=15645999
dynamic, 10000 run1=1.1520 run2=1.0860 run3=1.1660 avg=1.1260 s | thread busy min=1.1610 max=1.1660 (max/min=1.00) | max_steps=704 checksum=78178256 hits=15645999
guided run1=1.1770 run2=1.0960 run3=1.2110 avg=1.1535 s | thread busy min=1.2100 max=1.2100 (max/min=1.00) | max_steps=704 checksum=78178256 hits=15645999
N = 19009000, threads = 12, omp_get_max_threads = 12

=== Experiment A: False sharing ===
V1 naive hits[tid] threads=12 run1=0.8310 run2=0.7900 run3=0.8110 avg=0.8005 s | hits=15645999 | throughput=2.375e+07 iter/s
V2 padded struct threads=12 run1=0.6970 run2=0.7240 run3=0.7520 avg=0.7380 s | hits=15645999 | throughput=2.576e+07 iter/s
V2 reduction threads=12 run1=0.9550 run2=1.0050 run3=1.0050 avg=1.0050 s | hits=15645999 | throughput=1.891e+07 iter/s

=== Experiment B: Loop scheduling (12 threads) ===
static (default) run1=1.0220 run2=0.9930 run3=0.9530 avg=0.9730 s | thread busy min=0.6010 max=0.9520 (max/min=1.58) | max_steps=704 checksum=78178256 hits=15645999
static, 1000 run1=0.9540 run2=1.0100 run3=1.0430 avg=1.0265 s | thread busy min=0.6600 max=1.0410 (max/min=1.58) | max_steps=704 checksum=78178256 hits=15645999
dynamic, 100 run1=0.8990 run2=0.9390 run3=0.9150 avg=0.9270 s | thread busy min=0.9150 max=0.9150 (max/min=1.00) | max_steps=704 checksum=78178256 hits=15645999
dynamic, 10000 run1=0.8950 run2=0.9240 run3=0.8570 avg=0.8905 s | thread busy min=0.8510 max=0.8570 (max/min=1.01) | max_steps=704 checksum=78178256 hits=15645999
guided run1=0.8740 run2=0.8820 run3=0.8760 avg=0.8790 s | thread busy min=0.8370 max=0.8760 (max/min=1.05) | max_steps=704 checksum=78178256 hits=15645999
PS C:\Users\schan\paralcomp\amdahl_lab>
```

Figure 4: Output of collatz_phase4.exe with 8 threads and with 12 threads (logs: phase4_k8_log.txt, phase4_k12_log.txt).

Experiment A: False sharing

- **Variant 1 (naive):** a shared array `volatile int hit_count[256]`, aligned to 64 bytes, with `hit_count[omp_get_thread_num()]++` inside the loop. The volatile qualifier forces a real memory write on each increment so that the compiler cannot keep the counter in a register.
- **Variant 2 (mitigated):** a padded structure `struct Padded { int count; char pad[60]; }` aligned to 64 bytes, so each thread's counter occupies its own cache line. An OpenMP reduction(`+:total_hits`) variant was also measured as an extra.

Ratio = $T_{\text{variant}} / T_{\text{padded}}$. 8 threads is the physical core count; the 12-thread rows are given as a supplement.

Implementation variant	Threads	Exec time (s)	Throughput (iter/s)	Ratio vs padded (T / T_{padded})
Variant 1: naive <code>hits[tid]++</code>	8	1.0085	1.885e7	1.026
Variant 2: cache-padded struct	8	0.9825	1.935e7	1.000
Extra: OpenMP reduction	8	1.7730	1.072e7	1.805 (see note 1)
Variant 1: naive <code>hits[tid]++</code>	12	0.8005	2.375e7	1.085
Variant 2: cache-padded struct	12	0.7380	2.576e7	1.000
Extra: OpenMP reduction	12	1.0050	1.891e7	1.362 (see note 1)

Experiment B: Loop scheduling (12 threads, the maximum thread count)

Scheduling clause	Chunk size	Exec time (s)	Thread busy min / max (s)	max / min	Observed behavior
<code>schedule(static)</code>	Default (N / k)	0.9730	0.601 / 0.952	1.58	Uneven per-thread finish
<code>schedule(static, 1000)</code>	1,000	1.0265	0.660 / 1.041	1.58	Round-robin chunks did not fix the imbalance
<code>schedule(dynamic, 100)</code>	100	0.9270	0.915 / 0.915	1.00	Threads finish together
<code>schedule(dynamic, 10000)</code>	10,000	0.8905	0.851 / 0.857	1.01	Threads finish together
<code>schedule(guided)</code>	Exponential decay	0.8790	0.837 / 0.876	1.05	Nearly balanced; fastest overall

Measurement notes

- **Note 1:** The OpenMP reduction variant was expected to be at least as fast as the padded struct, since it involves no shared writes, but it was slower in both runs, and run 3 at 8 threads (2.262 s) was an outlier. I did not investigate the cause. It always ran third in the sequence, so thermal or power throttling is a possibility, but I did not test this. The padded struct is therefore used as the mitigated variant.
- **Note 2:** The 8-thread block of Experiment B (visible in Figure 4) ran slowly on the whole machine: `schedule(static)` took 2.7725 s, although the same 8-thread static loop took about 1.0 s in Experiment A and 1.3065 s in Phase 3. All eight threads were slow together (busy times 2.7 to 3.0 s), so this is a machine-wide slowdown rather than a scheduling effect. I did not investigate the cause and use the 12-thread block for the analysis. For completeness, the 8-thread averages were: static 2.7725 s, static/1000 2.6270 s, dynamic/100 1.2155 s, dynamic/10000 1.1260 s, guided 1.1535 s.

Phase 5: Technical Analysis

Verification

For $N = 19,009,000$ (from Student ID 230109009) every program, variant and schedule produced max stopping time = 704, checksum = 78178256 and 15,645,999 numbers with more than 100 steps. This is the correctness evidence for the parallel reductions.

Q1: Micro-architectural root cause of false sharing

My CPU's L1 cache line is 64 bytes. In Variant 1, the eight 4-byte counters `hit_count[0..7]` (32 bytes) share a single line. Each `hit_count[tid]++` needs exclusive ownership of the line, so under the MESI protocol (Intel uses a variant, MESIF) the writing core moves the line to the Modified state and invalidates the copies in the other cores' caches. The next core to write must fetch the line back. This repeated invalidation and transfer traffic is the slowdown, even though no two threads touch the same variable.

Measured effect: naive 1.0085 s vs padded 0.9825 s at 8 threads (1.026×), and 0.8005 s vs 0.7380 s at 12 threads (1.085×). The penalty was small rather than massive, probably because each iteration performs about 0.4 μs of Collatz computation per counter write (8 threads × 1.0085 s / 19.009 M iterations), so store buffers and out-of-order execution hide much of the transfer latency. The penalty grew with thread count because more cores contend for the same line.

Q2: Hyperthreading (SMT) saturation and physical core ceilings

No. Speedup did not continue to scale linearly. From 4 to 8 threads it rose from 3.74× to 6.49× (less than double); I did not measure thread placement, but this is likely because threads 5 to 8 run on the slower E-cores. From 8 to 12 threads the time fell from 1.3065 s to 0.9925 s, a 1.32× gain for 1.5× more threads. My i5-12450H has 8 physical cores and 12 logical threads (4 P-cores with SMT and 4 E-cores without), so the last four threads are SMT siblings sharing a core with another thread.

SMT siblings share the core's execution ports and ALUs, the L1 and L2 caches, the branch predictor, the decoders, the load/store buffers and the TLB. A second thread can fill pipeline stalls but cannot add execution hardware, so the gain is sub-linear. At $k = 16$, which exceeds the 12 logical threads, the time improved slightly to 0.887 s (1.12× over $k = 12$). My hypothesis is that the OS rebalances the uneven static blocks across cores, but I did not test it.

Q3: Empirical vs theoretical Amdahl discrepancy

The derived parallel fraction is $p = 2 \times [1 - 1/S_{\text{emp}}(2)] = 0.957$, using $T_1 = 8.4725$ s as the baseline (about 4.3% of the runtime is sequential). Using the separate sequential program instead ($T_{\text{seq}} = 8.9745$ s) gives $p = 1.016$, which is invalid and within measurement noise, so I deviated from the worksheet's T_{seq} baseline for the fit and state this explicitly.

The measured speedups (3.74×, 6.49×, 8.54×, 9.55× at $k = 4, 8, 12, 16$) stayed close to the fitted curve; Delta was slightly negative up to $k = 12$, which shows that p from a single $k = 2$ point is fragile. Both curves fall well below linear scaling because of the sequential fraction ($1 - p$). Two physical factors omitted by Amdahl's equation:

- **Heterogeneous cores:** the CPU has 4 fast P-cores and 4 slower E-cores. With static scheduling each thread gets an equal block of iterations, so fast cores finish early and wait (busy time max/min = 1.58 at 12 threads), and extra threads add less than a full P-core's throughput.
- **SMT resource sharing plus turbo/thermal limits:** the last four threads compete for shared execution units, and per-core clock speed drops as more cores are active on a laptop power budget.

Other omitted factors include cache and memory contention and OS scheduling effects.

Q4: The scheduling trade-off dilemma (12 threads)

Static took 0.973 s, static/1000 1.0265 s, dynamic/100 0.927 s, dynamic/10000 0.8905 s and guided 0.879 s. The static schedules left threads finishing unevenly (busy time max/min = 1.58, even with chunk 1000), which points to hardware speed differences between cores, since the chunks contain similar amounts of Collatz

work. Dynamic and guided schedules had max/min between 1.00 and 1.05 and took about 5 to 10% less time than static.

Dynamic(100) was about 4% slower than dynamic(10000), consistent with more queue operations (about 190,000 chunk grabs versus 1,900), but it still beat static. The differences among dynamic(10000), guided and dynamic(100) are close to my run-to-run noise (dynamic/10000 alone ran 0.924 s and 0.857 s), so I cannot rank them firmly. I found no chunk size in the tested range where lock-queue contention outweighed the balancing benefit.

Submission Contents

Archive lab1_230109009.tar.gz contains: hw_info.txt, collatz_seq.c, collatz_par.c, collatz_phase4.c, results.csv (Tables 1 to 3 raw data), speedup_plot.png, the raw logs (seq_log.txt, phase3_log.txt, phase4_k8_log.txt, phase4_k12_log.txt) and this document as analysis.pdf.